



기존 SSM의 문제

SSM은 다음처럼 상태를 갱신합니다.

$$h_t = Ah_{t-1} + Bx_t$$

여기서

- A : 기존 기억을 얼마나 유지할지
- B : 새로운 입력을 얼마나 기억에 넣을지
- C : 기억에서 무엇을 출력할지

를 결정합니다.

그런데 기존 SSM에서는 A, B, C 가 입력과 관계없이 고정되어 있습니다.

예를 들어 친구의 말을 들으며 메모한다고 해보겠습니다.

- “내일 시험 범위는 3장까지야.” → 중요한 정보
- “오늘 점심은 김밥을 먹었어.” → 덜 중요한 정보

기존 SSM은 두 문장을 구분하지 못하고 항상 동일한 규칙으로 메모합니다.

즉,

중요한 정보도 50% 기록하고, 중요하지 않은 정보도 50% 기록하는 식입니다.

입력 내용을 보고 판단하는 것이 아니라, 미리 학습된 고정된 방식으로 모든 입력을 처리합니다.

LSTM과 비교하면

LSTM은 입력을 볼 때마다 게이트를 계산합니다.

- 이 정보는 중요하니까 많이 기억
- 이 정보는 필요 없으니까 삭제
- 이 정보는 지금 출력에 사용

따라서 LSTM은 입력에 따라 기억 방법을 바꿀 수 있습니다.

하지만 이 게이트가 이전 상태에 의존하는 비선형 계산이기 때문에,

$$h_1 \rightarrow h_2 \rightarrow h_3$$

순서대로 계산해야 합니다. 그래서 학습 병렬화가 어렵습니다.

반면 기존 SSM은 기억 규칙이 단순하고 고정되어 있어 병렬 계산은 쉽지만, 입력별 선택 능력이 부족합니다.

모델	입력별 중요도 판단	학습 병렬화
LSTM	가능	어려움
기존 SSM	어려움	가능
Mamba	가능	가능

그래서 Mamba가 무엇을 바꿨는가

Mamba는 기존 SSM의 고정된 B, C 등을 현재 입력에 따라 바뀌도록 만들었습니다.

개념적으로는 다음과 같습니다.

$$B_t = f_B(x_t), \quad C_t = f_C(x_t)$$

즉, 모든 시점에 같은 B, C 를 사용하는 것이 아니라 입력 x_t 를 보고 그때그때 결정합니다.

예를 들면 다음과 같습니다.

- “시험 범위는 3장까지야”
→ B_t 를 크게 설정해서 상태에 많이 기록
- “점심은 김밥이었어”
→ B_t 를 작게 설정해서 거의 기록하지 않음
- 현재 답변에 필요한 정보
→ C_t 를 조절해 상태에서 적극적으로 꺼냄

이것을 Mamba 논문에서는 선택성, **Selectivity**라고 부릅니다.

메모지 비유로 전체 정리

RNN/LSTM

메모지 크기는 고정되어 있지만, 새 문장을 들을 때마다 판단합니다.

“이건 중요하니까 적고, 이건 필요 없으니까 지우자.”

선택은 잘하지만 이전 메모를 먼저 읽어야 다음 메모를 쓸 수 있어서 순차적입니다.

기존 SSM

메모지 크기는 고정되어 있고, 메모 규칙도 고정되어 있습니다.

“무슨 말이 나오든 항상 같은 비율로 적자.”

전체 계산을 병렬화하기는 좋지만 중요한 말과 중요하지 않은 말을 구분하기 어렵습니다.

Mamba

메모지 크기는 고정되어 있지만, 입력에 따라 메모 규칙을 바꿉니다.

“이 문장은 중요하니 많이 적고, 이 문장은 필요 없으니 건너뛰자.”

즉, **LSTM**의 선택 능력과 **SSM**의 효율적인 계산을 결합하려는 구조입니다.

세 가지 차이로 정리

1. 기억 크기

LSTM과 Mamba 모두 고정된 크기의 상태에 과거 정보를 압축합니다. Transformer처럼 이전 토큰 전체를 저장하고 매번 참조하지 않습니다.

2. 기억의 선택성

기존 SSM은 모든 입력을 거의 같은 규칙으로 처리하지만, Mamba는 입력을 보고 무엇을 기억하고 잊을지 선택합니다.

3. 계산 방식

LSTM은 순서대로 계산해야 하지만, Mamba는 선택적 상태 업데이트를 효율적인 병렬 스캔 방식으로 구현해 학습 속도를 높입니다.

한 문장으로 정리하면 다음과 같습니다.

Mamba는 고정된 크기의 상태에 정보를 압축하면서, 입력마다 중요한 정보만 선택적으로 기억하고, 이를 병렬적으로 효율 있게 계산하는 모델입니다.

완전히 다른 종류의 기억 공간은 아닙니다.

Mamba의 상태 h_t 도, RNN/LSTM의 hidden state나 cell state도 모두 과거 정보를 고정 크기 벡터에 압축한 내부 기억입니다.

차이는 주로 그 기억을 업데이트하는 수학적 방식에 있습니다.

1. RNN의 기억 공간

RNN은 보통 다음처럼 상태를 갱신합니다.

$$h_t = \tanh(W_h h_{t-1} + W_x x_t)$$

이전 기억 h_{t-1} 과 현재 입력 x_t 을 합친 뒤 비선형 함수에 넣습니다.

즉, 기억 업데이트가 다음과 같습니다.

이전 기억 + 현재 입력 → 비선형 변환 → 새로운 기억

이 과정은 이전 상태를 실제로 계산해야 다음 상태를 계산할 수 있습니다.

$$h_1 \rightarrow h_2 \rightarrow h_3$$

따라서 순차적입니다.

2. LSTM의 기억 공간

LSTM은 기억을 사실상 두 개로 나눕니다.

c_t : 장기 기억

h_t : 현재 출력에 가까운 단기 표현

핵심 기억인 cell state는 다음처럼 업데이트됩니다.

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

여기서 입력에 따라 게이트를 계산합니다.

- f_t : 이전 정보를 얼마나 잊을지
- i_t : 새로운 정보를 얼마나 넣을지

- o_t : 기억을 얼마나 출력할지

따라서 LSTM은 입력별로 매우 유연하게 기억을 조절할 수 있습니다.

하지만 게이트가 h_{t-1} 에 의존하므로 역시 순차 계산해야 합니다.

3. Mamba의 상태 공간

Mamba의 핵심 상태 업데이트는 기본적으로 SSM 형태입니다.

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t$$

$$y_t = C_t h_t$$

여기서 h_t 가 실제 기억 벡터입니다.

LSTM과 달리 상태 변화의 중심이 구조화된 선형 동역학입니다.

- $\bar{A}_t$: 이전 기억을 어떻게 감쇠하거나 유지할지
- $\bar{B}_t$: 현재 입력을 기억에 어떻게 기록할지
- C_t : 기억에서 무엇을 읽을지

Mamba에서는 $\bar{B}_t, C_t$, 그리고 시간 간격에 해당하는 Δ_t 가 입력에 따라 달라집니다. 그래서 중요한 입력은 오래 남기고, 불필요한 입력은 빨리 잊을 수 있습니다.

핵심 차이

구분	RNN	LSTM	Mamba
실제 기억	hidden state h_t	cell state c_t , hidden state h_t	SSM state h_t
기억 크기	고정	고정	고정
업데이트 방식	비선형 반복	비선형 게이트	구조화된 선형 상태 전이 + 입력 의존적 선택
입력별 선택	제한적	게이트로 선택	입력에 따라 기록·유지·출력 선택
학습 계산	순차적	순차적	병렬 스캔 가능
긴 시퀀스 처리	정보 소실 가능	RNN보다 개선	긴 범위의 감쇠와 유지가 구조적으로 설계됨

가장 중요한 개념적 차이

LSTM

기억 공간의 각 값에 대해 직접 게이트를 적용합니다.

이 값을 30% 잊고, 새 정보를 70% 넣자.

Mamba

상태를 하나의 동적 시스템처럼 봅니다.

이 입력이 들어왔을 때 내부 상태가 어떤 속도로 변하고, 어떤 정보가 얼마나 오래 남도록 할 것인가?

즉, LSTM은 메모리 셀을 게이트로 관리하는 방식이고, Mamba는 동적 시스템의 상태 변화를 이용해 기억을 관리하는 방식입니다.

다만 Mamba의 선택 메커니즘은 결과적으로 LSTM의 게이트와 비슷한 역할을 합니다.

한 문장으로 정리하면:

RNN/LSTM과 Mamba 모두 고정 크기 벡터에 과거를 저장하지만, LSTM은 비선형 게이트로 기억을 직접 조절하고, Mamba는 구조화된 상태 전이와 입력 의존적 파라미터로 기억의 유지·기록·출력을 조절합니다.